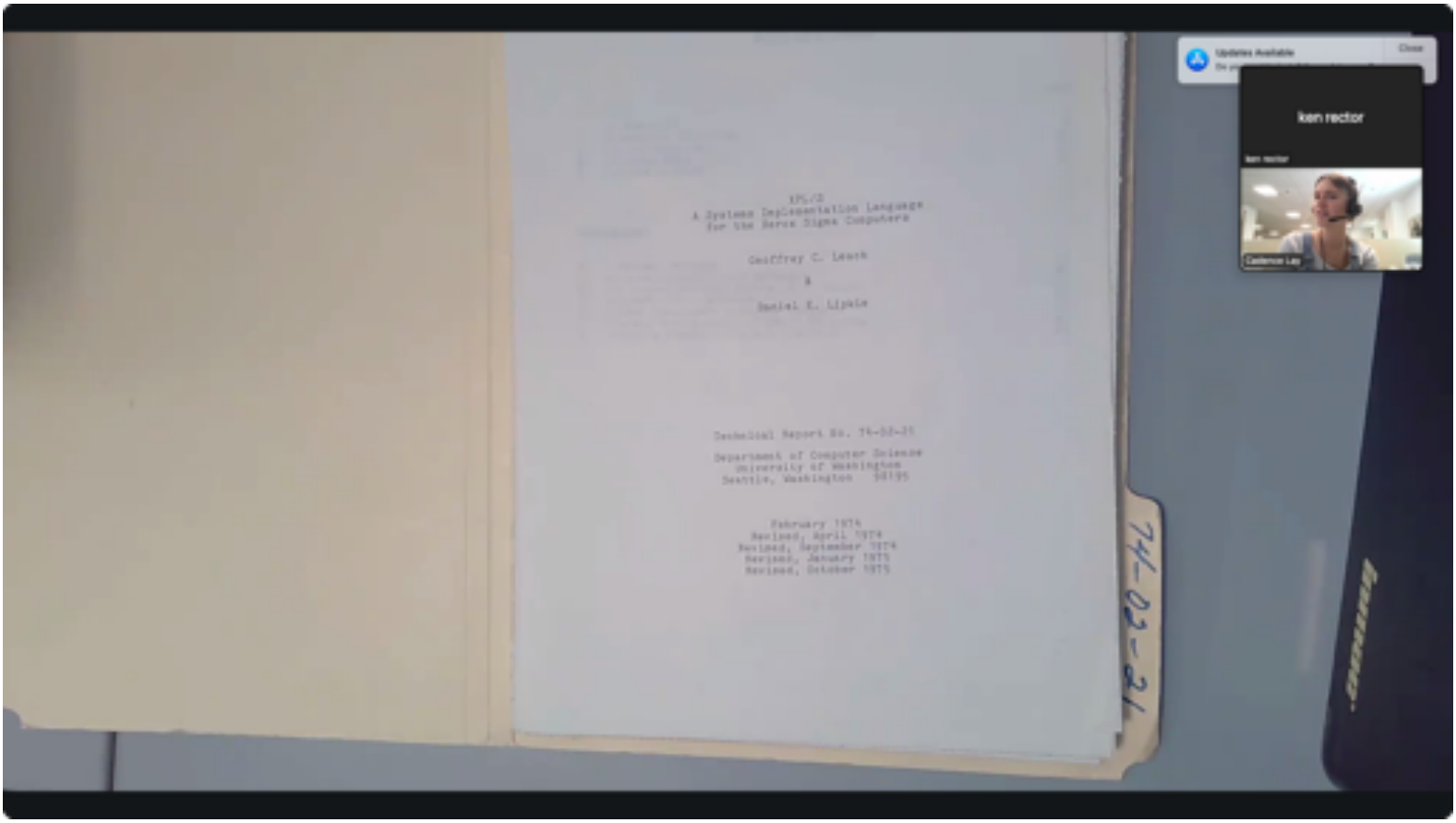

XPL/2
A System Implementation Language
for the Perq Sigma Computers
Geoffrey C. Lewis
&
Daniel E. Lipkie
Technical Report No. 74-02-21
Department of Computer Science
University of Washington
Seattle, Washington 98195
February 1974
Revised, April 1974
Revised, September 1974
Revised, January 1975
Revised, October 1975
74-02-21
Updates Available
Close
ken rector
ken rector
Cadence Lee

XPL/S
A Systems Implementation Language for the Xerox Sigma Computers

Geoffrey C. Leach

&

Daniel E. Lipkie

Technical Report No. 74-02-21

Department of Computer Science
University of Washington
Seattle, Washington 98195

February 1974
Revised, April 1974
Revised, September 1974
Revised, January 1975
Revised, October 1975

74-02-21

Table of Contents

		Page
		1
1.	An Overview	7
2.	Syntax and Semantics	20
3.	Lexical Entities	23
4.	Built-in Functions	28
5.	Runtime Support	

Appendicies

A.	Compiler Messages	33
B.	Runtime Support Error Messages	37
C.	BNF Syntax for the Simple LR(1) Parser	39
D.	Syntactic Metalanguage	43
E.	Coding Compatible Assembly Language	44
F.	Storage Organization of XPL/S Programs	47
G.	Separate Compilations and Overlays	48

XPL/S Technical Report

1. An Overview

In September, 1972, an implementation of the language XPL, as described by McKeeman, Horning and Wortman in their book A Compiler Generator [1] was completed at the University of Washington for the XEROX Sigma 5 computer. This work, which has been described elsewhere [2,3,4], culminated in a language implementation, XPL/S Sigma, which differed from the original implementation, XPL/360 only in minor details. This was the result of two factors: the similarity of the S/360 and Sigma architectures and the determination of the implementors to avoid excursions into language extension.

During the 1972-73 academic year, the language received only moderate usage, despite the fact that, objectively, XPL was much more suited to many of the projects undertaken on the Sigma 5 than the languages actually used. It was observed that persons contemplating smaller projects (to be completed in less than four weeks) tended to give no consideration at all to the choice of a language. The one used was generaly chosen from among those with which they were already familiar. Those contemplating projects of larger scope were prepared to consider learning a new language if there was a reasonable probability that it would reduce the total time spent on the project. Such persons were, as a rule, more experienced programmers. As a result, there was considerable informed feedback as to language design and implementation. The most common theme was that XPL/Sigma failed to take advantage of its environment, that is, that it was not well adapted to the available hardware and software.

The changes (both additions and deletions) that have resulted from this period of examination and described in this section. Also described in this section are the changes made in the language during the 1974-75 academic year to allow separate compilation and overlaying of XPL/S programs. In this report, XPL refers to the original (/360) language and XPL/S to the current (Sigma) version.

Control Structures

Recently, considerable attention has been given to the effect of the control structures available in a language on the style of programming. This has become known as the "goto controversy" [5]. In the light of this, an informal study of XPL usage was made. This revealed, perhaps not surprisingly, that those who made use of the goto statement to any extent tended to do so in an entirely unstructured manner. Consequently, the standards committee (membership of one) decided, with the missionary zeal that is well known to be characteristic of such bodies, to replace the goto with control structures that would, hopefully, encourage better programming style. The additions chosen were an exit statement and an end until statement. Along with removal of the goto syntax, the label data type and the ability to make forward references to procedures have also been removed. Labels may appear only on group heads and as procedure names.

The first control construct to be added was the exit statement. This causes a branch to the instruction following the end of the

XPL/S Technical Report

1. An Overview

In September, 1972, an implementation of the language XPL, as described by McKeeman, Horning and Wortman in their book A Compiler Generator [1] was completed at the University of Washington for the XEROX Sigma 5 computer. This work, which has been described elsewhere [2,3,4], culminated in a language implementation, XPL/S Sigma, which differed from the original implementation, XPL/360 only in minor details. This was the result of two factors: the similarity of the S/360 and Sigma architectures and the determination of the implementors to avoid excursions into language extension.

During the 1972-73 academic year, the language received only moderate usage, despite the fact that, objectively, XPL was much more suited to many of the projects undertaken on the Sigma 5 than the languages actually used. It was observed that persons contemplating smaller projects (to be completed in less than four weeks) tended to give no consideration at all to the choice of a language. The one used was generaly chosen from among those with which they were already familiar. Those contemplating projects of larger scope were prepared to consider learning a new language if there was a reasonable probability that it would reduce the total time spent on the project. Such persons were, as a rule, more experienced programmers. As a result, there was considerable informed feedback as to language design and implementation. The most common theme was that XPL/Sigma failed to take advantage of its environment, that is, that it was not well adapted to the available hardware and software.

The changes (both additions and deletions) that have resulted from this period of examination and described in this section. Also described in this section are the changes made in the language during the 1974-75 academic year to allow separate compilation and overlaying of XPL/S programs. In this report, XPL refers to the original (/360) language and XPL/S to the current (Sigma) version.

Control Structures

Recently, considerable attention has been given to the effect of the control structures available in a language on the style of programming. This has become known as the "goto controversy" [5]. In the light of this, an informal study of XPL usage was made. This revealed, perhaps not surprisingly, that those who made use of the goto statement to any extent tended to do so in an entirely unstructured manner. Consequently, the standards committee (membership of one) decided, with the missionary zeal that is well known to be characteristic of such bodies, to replace the goto with control structures that would, hopefully, encourage better programming style. The additions chosen were an exit statement and an end until statement. Along with removal of the goto syntax, the label data type and the ability to make forward references to procedures have also been removed. Labels may appear only on group heads and as procedure names.

The first control construct to be added was the exit statement. This causes a branch to the instruction following the end of the

nearest enclosing group. Optionally, a label may be designated which causes the branch to be made to the instruction following the end of the enclosing group whose do carries the label. Exiting to procedure labels or exiting out of the scope of a procedure is not permitted.

The second control construct to be added was the end until statement which is an addition to the syntax of group endings. If end is followed by until <expression> then the group is repeated until the least significant bit of the expression is a 1. The construct may be used in conjunction with do ... to, but has no meaning in conjunction with do case.

These structures make it easier for programmers who are accustomed to programming with the goto to live without it. They do not enforce any particular style; individuals tend to adapt their old styles to the new constructs. One unexpected side effect has been encountered, however. Some persons, familiar only with FORTRAN, have expressed a fear of being unable to program without the goto, and were consequently unwilling to even attempt to learn XPL/S.

In order to facilitate error checking, both by the programmer and by the compiler, the otherwise statement has been added. If the end corresponding to a do case is followed by otherwise <basic statement>, the basic statement is executed if the index of the case is out of range.

External communication

XPL has no facility for the description of external procedures or data types. In fact, the compiler produces absolute code which requires only a minimal interface with the host operating system. This drawback has been overcome in two stages.

Initially, the compiler interface with the host system was made through the loader. That is, compiler output was relocatable object language to be processed into absolute form by the loader. The immediate advantage of this is that the compiler may be simplified in two respects: treatment of forward references and generation of code for runtime support functions. The former are handled with loader directives, the latter with external references, satisfied at load time from the system library.

This facility is also made available to the user of XPL/S through syntactic extensions. The external procedure facility is available by prefixing external to the reserved word procedure, specifying the procedure name, specifying and declaring parameters, if any, and concluding with an <ending>. external procedures may be defined in assembly language or XPL/S. The global facility is available by prefixing global to the reserved word procedure, specifying the procedure name, specifying and declaring parameters, if any, defining the <statement list>, and concluding with an <ending>. global procedures may be accessed from any compilation in which there is a corresponding external procedure declaration. The FORTRAN procedure facility is available by prefixing fortran to the reserved word procedure, specifying the procedure name, specifying the parameters, if any, and concluding with an <ending>. Declaration of the

74-02-21

parameters is not necessary since they are passed by reference as fixed variables.

External data is declared using the normal declaration with the reserved word external replacing the optional <initial list>. Global data is declared by prefixing global to the optional <initial list>, and may be accessed from any compilation in which there is a corresponding external declaration. Formal parameters are passed by value if the normal data declaration is used, or by reference if the reserved word ref replaces the <initial list>. Data may be passed by reference within a compilation or between separate compilations if both the external and global procedures declare the parameter ref.

The introduction of these features permits the XPL/S programmer to make access to facilities outside the language. In particular, two areas of interest have developed: communication with unsupported devices, especially graphics terminals, and floating point computation. Inner loop optimization is another possibility, but this has not received any attention as yet.

Improved access to hardware

One of the original design objectives of XPL was to have a higher level language that was, in some sense, close to the machine. This has been continued in the provision of access to certain features of the Sigma architecture not found in the 360 series. These are: a data type that uses the stack manipulation hardware, expansion of the repertoire of shift functions, addition of ABS and shift built-in functions and an exclusive-or operator.

The available stack hardware is limited to load/store with accompanying stack pointer manipulation. This is utilized through a stack data type which causes the initialization of an appropriate stack pointer double word [6]. The size of the stack is obtained from the declared dimension, hence there is no array of stacks. The semantics are what would be expected; appearance of a stack identifier on the left of the assignment operator causes a push, on the right causes a pull. If the identifier appears with a subscript then the access is treated as a normal array operation, with the zeroth word being of the stack pointer doubleword. Reserved words underflow and overflow have been added to the syntax for <logical primary> to permit testing for those conditions. As utilized by the stack pointer doubleword, the hardware generates condition code patterns when these occur. The user may also initialize his stack in such a way as to cause program abort on either or both.

The shift functions ASHL, ASHR, CSHL and CSHR were added to permit the generation of arithmetic and circular shift instructions in the same way as SHL and SHR generate logical shifts. ABS generates a load absolute.

The exclusive-or operator, xor uses that instruction in the Sigma operation set. xor has the same precedence as the inclusive-or operator, !.

Optimizations and semantics extensions

Wetherell [7] has pointed out that the replacement of <number> by <expression> in the syntax for <bit head>, <bound head> <initial head>, together with the inclusion of a facility for compile time evaluation of numeric expressions with constant terms, provide a useful flexibility in the generation of declarations. This has been implemented, with extensions where possible to the built-in functions.

Techniques implemented by Bolstad and Wetherell in the Lawrence Livermore Laboratories XPL to optimize code generation in certain cases have also been adopted. In general, the technique used is to use the commutativity of operations and the availability of byte and halfword load/store operations to replace two loads plus a register to register operation by a single load followed by a storage to register operation.

Stack and syntactic modifications suggested in [7] have been used to make concatenation of strings recursive. Previously, if evaluation of the second operator to || itself involved concatenation, the first operand would be lost. The availability of the stack instructions mean that this is not significantly more expensive than the non-recursive implementation.

An interpretation of strings appearing in the initialization of non-character variables has been added. The appropriate number of characters are used from the left of the string and stored as EBCDIC. Bit strings appearing in the initialization of character variables are treated as binary strings irrespective of their length. This is especially useful in programming control transmission for interactive terminals.

It will be observed that the syntax does not forbid the appearance of procedure identifiers on the left of the assignment operator. We have added semantics whereby this is interpreted as a call with the expression to the right of the assignment as an implicit final parameter. Thus, the compiler need not treat INPUT and OUTPUT as special cases.

A number of library routines are provided to the XPL/S user. These are primarily concerned with establishing an integrated environment for tape/disk input-output, which was absent in XPL. These are REWIND, UNLOAD, WEOF (write end-of-file), INPUTA, OUTPUTA (for binary input-output), TEST_EOF (for end-of-file tests, particularly in binary IO), SFILEF, SFILEB (skip file forward and backward), PFILEF and PFILEB (position file at bof and eof) and PREC (position record). Together these provide sufficient capabilities to replace the FILE pseudoarray, which has been removed from the language.

Input-output units are limited to three in XPL. Inevitably, this was found to be either too many (in terms of the required control blocks) or too few. An ingenious suggestion by Duane Sand has been implemented instead. XPL/S provides default declaration only for the card reader and line printer (XPL unit 1). Any other device may be accessed by passing the name of the control block to INPUT or OUTPUT.

The control block itself is named and created at load time. Thus, INPUT(1) becomes INPUT('M:SI'), where M:SI is the name of the standard card reader control block.

Separate Compilation and Overlays

XPL/S procedures and data may be separately compiled and built into an overlay structure using the operating system loader. In the program load module, control is passed by calls to global procedures and data is passed by parameters and references to external and global data. Compilations may be any combination of data declarations and procedure definitions, (e.g. "pure data" compilations of global data). However only one compilation per program is allowed to have executable code generating statements outside a procedure definition. This main compilation defines the program's entry point.

A built-in procedure loads overlay segments on request. Overlays must be loaded in root to leaf order and segments loaded must re-establish communication to a descendent by reloading all segments along the overlay path before the descendent may be called. Loading a segment will reset all its variables to their initial values.

Plan of the Report

The following section presents the syntax and semantics of XPL/S. The semantics are presented in the order of productions and are indexed by a condensed syntax, which appears as the first page of the section. The syntactic metalanguage is adapted from one devised by Professor Robert G. Herriot of the University of Washington and has many other antecedents. A description of the metalanguage appears in Appendix C. The BNF Syntax that was used to produce the syntax tables used by the simple LR(1) parser [8] is provided in Appendix C.

Error messages that may be produced by the compiler are listed in Appendix A. These are the diagnostics actually occuring in the syntatic and semantic analysis of the XPL/S source program. Those that have their origin in the runtime support are given in Appendix B. They are common to all XPL/S programs and may occur in the course of execution of any XPL/S program, including the compiler.

The error checking capabilities of the compiler are not highly developed. Syntax errors are flagged and the user must puzzle out for himself just what the compiler was expecting. Semantic errors have a more specific explanation but, in most cases no default interpretation is provided. Except in cases of drastic table overflow compilation is permitted to continue to the end of the program and loading is not inhibited.

The formation of identifiers and other lexicographic considerations are discussed in Section 3. Section 4 deals with a special class of identifiers, the built-in (or implicitly declared) functions.

Acknowledgements

The language described in this report and the compiler that implements it have benefited considerably from the suggestions of many individuals. In particular, the authors would like to extend their appreciation to Duane Sand, John Rieber, Donn Terry, and William Mallgren of the Department of Computer Science, University of Washington, William Lederer of Telemed, Inc. and Charles Codling of the Xerox Corporation. Particular credit is due to Robert A. Lovestedt who designed and implemented the otherwise statement and the fortran procedure calling sequence. XPL/S through version B00 is principally the result of the work of Geoffrey C. Leach. The work was supported in part by National Science Foundation Grant GJ 28781. XPL/S version C00 is the result of the work by Daniel E. Lipkie.

References

1. W.M. McKeeman, J.J. Horning, and D.B. Wortman, A Compiler Generator. Prentice-Hall, Englewood Cliffs, New Jersey, 1970.

2. G.C. Leach, "The University of Washington XPL Compiler" Proceedings of the 19th International Meeting of th XDS User's Group, II, (December, 1972) 89-101.

3. G.C. Leach, The University of Washington XPL Compiler, University of Washington Department of Computer Science Technical Report 72-12-06, Seattle, Washington (December, 1972).

4. G.C. Leach and N. Golde, "Bootstrapping XPL to an XDS Sigma 5 Computer", Software-Practice and Experience 3, 235-244 (1973).

5. B.M. Leavenworth et al. "The Goto Controversy" Sigplan Notices 8,11 (November, 1972).

6. Xerox Data Systems, XEROX Sigma 5 Reference Manual.

7. C. Wetherell, XPL Users Manual Lawrence Livermore Laboratory, Livermore, California, February, 1973.

8. Franklin DeRemer, "Simple LR(k) Grammars" Communications of the ACM 14,7 (July, 1971) 453-360.

2. Syntax and Semantics

Page

1. prog -> { stmt }+ 8
2. stmt -> basic-stmt | if-stmt 8
3. if-stmt -> if exp then { basic-stmt else } stmt 8
4. basic-stmt -> { group | proc | call | return | exit | assgt | decl }; 8
5. group -> { LABEL-ident } do { var = exp to exp { by exp } | while exp | case exp }; { stmt }* end { LABEL-ident | until exp | otherwise basic-stmt } 9
6. proc -> {external | fortran | global} procedure PROC-ident { ({ , . FORMAL-ident}+)} { fixed | stack | character | bit (CONST-exp) | floating }; { stmt } + end { PROC-ident } 10
7. call -> PROC-var
8. return -> return { exp } 11
9. exit -> exit { LABEL-ident } 11
10. assgt -> { , . var }+ = exp 12
11. decl -> declare { , . { ident literally string | {{ ident | ({ , . ident }+) } { (CONST-exp) } { fixed | stack | character | bit (CONST-exp) | floating } { [global] initial ([,. CONST-exp]+) | ref | external }}}} 12 13
12. exp -> { | | xor | & | . { ¬ } log-prim }+
13. log-prim -> {{= | > | < | >= | <= | ¬= | ¬< }. str-exp }+ | underflow | overflow } 17 17
14. str-exp -> { str-exp || . arith-exp }+
15. arith-exp -> { + | - | * | / | mod . {+|-} arith-primary }+ 18
16. arith-primary -> var | (arith-exp) | number | string 18
17. var -> ident { ({ , . exp }+) } 18 18

1. prog -> [stmt]+

An XPL/S program is defined as a sequence of one or more statements. This sequence is terminated by the detection of an end-of-file on the source medium.

2. stmt -> basic-stmt | if-stmt

The dichotomy between if and basic statements is made for the purpose of avoiding ambiguity of association of then...else pairs within nested if statements.

3. if-stmt -> if exp then [basic-stmt else] stmt

The if statement provides for conditional execution based on a test. Any expression may appear as 'exp'; that is, there are no Boolean expressions in the language. However, the expression must be numeric. The determination of true or false is made on the basis of the least significant (rightmost) bit of exp. If this bit is a 1, then the statement following the then is executed. If it is a 0, the statement following the then is skipped and the one following the else (if any) is executed.

Note that if no else appears, any statement may follow the then. However, if else is used, then only a basic statement, that is, any statement but an if statement, may follow the then.

The following statements are legal.

```
IF A < B THEN A = B;  ELSE B = A;
IF A < B THEN A = B;  ELSE IF A = B THEN B = A;
```

However, it is not correct to write:

```
IF A < B THEN IF A = 1 THEN A = B;  ELSE B = C;
```

Rather, we must write:

```
IF A < B THEN DO;
   IF A = 1 THEN A = B;  ELSE B = A;
END;  ELSE B = C;
```

4. basic-stmt -> [group | proc | call | return | exit | assgt
| decl];

This production enumerates the various basic statements. Note that there is an empty statement. All statements are terminated with a ';'. The procedure and declaration statements may appear anywhere. However, the declaration statement does not generate any executable code and the procedure statement code is not executed until it is called.

1. prog -> [stmt]+

An XPL/S program is defined as a sequence of one or more statements. This sequence is terminated by the detection of an end-of-file on the source medium.

2. stmt -> basic-stmt | if-stmt

The dichotomy between if and basic statements is made for the purpose of avoiding ambiguity of association of then...else pairs within nested if statements.

3. if-stmt -> if exp then [basic-stmt else] stmt

The if statement provides for conditional execution based on a test. Any expression may appear as 'exp'; that is, there are no Boolean expressions in the language. However, the expression must be numeric. The determination of true or false is made on the basis of the least significant (rightmost) bit of exp. If this bit is a 1, then the statement following the then is executed. If it is a 0, the statement following the then is skipped and the one following the else (if any) is executed.

Note that if no else appears, any statement may follow the then. However, if else is used, then only a basic statement, that is, any statement but an if statement, may follow the then.

The following statements are legal.

```
IF A < B THEN A = B;  ELSE B = A;
IF A < B THEN A = B;  ELSE IF A = B THEN B = A;
```

However, it is not correct to write:

```
IF A < B THEN IF A = 1 THEN A = B;  ELSE B = C;
```

Rather, we must write:

```
IF A < B THEN DO;
   IF A = 1 THEN A = B;  ELSE B = A;
END;  ELSE B = C;
```

4. basic-stmt -> [group | proc | call | return | exit | assgt
| decl];

This production enumerates the various basic statements. Note that there is an empty statement. All statements are terminated with a ';'. The procedure and declaration statements may appear anywhere. However, the declaration statement does not generate any executable code and the procedure statement code is not executed until it is called.

1. prog -> { stmt }+

An XPL/S program is defined as a sequence of one or more statements. This sequence is terminated by the detection of an end-of-file on the source medium.

2. stmt -> basic-stmt | if-stmt

The dichotomy between if and basic statements is made for the purpose of avoiding ambiguity of association of then...else pairs within nested if statements.

3. if-stmt -> if exp then [basic-stmt else] stmt

The if statement provides for conditional execution based on a test. Any expression may appear as 'exp'; that is, there are no Boolean expressions in the language. However, the expression must be numeric. The determination of true or false is made on the basis of the least significant (rightmost) bit of exp. If this bit is a 1, then the statement following the then is executed. If it is a 0, the statement following the then is skipped and the one following the else (if any) is executed.

Note that if no else appears, any statement may follow the then. However, if else is used, then only a basic statement, that is, any statement but an if statement, may follow the then.

The following statements are legal.

```
IF A < B THEN A = B;  ELSE B = A;
IF A < B THEN A = B;  ELSE IF A = B THEN B = A;
```

However, it is not correct to write:

```
IF A < B THEN IF A = 1 THEN A = B;  ELSE B = C;
```

Rather, we must write:

```
IF A < B THEN DO;
   IF A = 1 THEN A = B;  ELSE B = A;
END;  ELSE B = C;
```

4. basic-stmt -> [group | proc | call | return | exit | assgt
| decl];

This production enumerates the various basic statements. Note that there is an empty statement. All statements are terminated with a ';'. The procedure and declaration statements may appear anywhere. However, the declaration statement does not generate any executable code and the procedure statement code is not executed until it is called.

5. group -> [LABEL-ident] do [var = exp to exp [by exp]
| while exp | case exp];
{ stmt }*
end [LABEL-ident | until exp
| otherwise basic-stmt]

The optional LABEL-ident that may precede the do at the head of the group serves as the identifier of the group for exit statements that may appear within it. Note: the label is not the target of the exit; see production 9.

Four types of header on the group are permitted; simple, iteration, while and case.

Simple header: do; This merely marks the beginning of the group and serves as a place to attach a label.

Iteration header: do var = exp1 to exp2 [by exp3];; when the group is entered, var (which may not be subscripted) is set to exp1. If this value does not exceed exp2 the body of the group, { stmt }+, is executed. On subsequent iterations the value of var is incremented by exp3, or by 1 if by exp3 is omitted. Exp2 and exp3 are evaluated once at the start of the loop and are not affected by subsequent changes in their components. Exp3 must be positive, or the group will never terminate.

While header: while exp; The group is executed if, and while, the least significant bit of exp is 1. Exp is evaluated before each execution of the group.

Case header: case exp; exp is evaluated and the exp-th statement of the group body (counting from zero) is executed. If exp < 0 or exp exceeds the number of statements. The otherwise statement will be executed, if present. If not present the program will abort with an error message.

A group may terminate with end, followed optionally by the label that appears on the group header. This serves to identify corresponding do ... end pairs. Alternatively, a group may terminate with end until exp. This is the analog of the while header; exp is evaluated and if the least significant bit is zero, a branch to the head of the group body is made, otherwise the group terminates. end until is permitted on all types of group but has no meaning, and is never executed, for case groups. The otherwise ending is permitted only on code groups.

The following examples illustrate some of the uses of the various types of group.

```
DO I = 5 TO 10 BY STEP;
    .
    .
    .
END;
```

```
E DO WHILE A<B;
    .
    .
    .
  END UNTIL B>5;

C DO CASE I;
    STMT0;
    STMT1;
    .
    .
    .
  END C;

DO CASE J;
  STMT0;
  STMT1;
    .
    .
    .
END OTHERWISE STMTx;
```

6. proc -> [external | fortran | global] procedure PROC-ident
[((, . FORMAL-ident)+)]
[fixed | stack | character | bit (CONST-exp)
| floating]+ ;
[stmt]+
end [PROC-ident]

The procedure statement defines an XPL/S procedure. Procedure statements may appear in any place that a statement is allowed. However the code is only executed when invoked through a call; that is, it is not executed in line.

An external procedure is a procedure whose code is defined external to the XPL/S program being compiled. A fortran procedure is similar to an external procedure but the standard FORTRAN subroutine linkage is used. This means all parameters are passed by reference. If it is desired to call floating point FORTRAN functions the floating type must be used in the declaration. It is the user's responsibility to call 9INITIAL if it is required. A global procedure is a procedure whose code is defined in this compilation and may be called from within this compilation or any compilation in which it is declared as an external procedure.

PROC-ident is the name by which the procedure is known. Its appearance causes a new scope to be opened. All identifiers declared within the procedure statement are local to it, and may not be referenced outside.

Next, the (optional) formal parameters appear. If there are any, they are a series of identifiers, separated by commas. The formal parameter identifiers must be unique.

Following the formal parameters, the data type returned, by the procedure, if any, is specified. This is largely for documentation. Fixed, stack and short bit string types are treated alike when used as functions.

In the body must occur declarations for formal parameters. Only declarations may occur in the body of external and fortran procedure declarations. All fortran procedure formal parameters are fixed ref and the declarations may be omitted but the <ending> is required. The name of the procedure may appear within its body, but note that recursion is not supported. The user must make his own arrangements (see Built-Ins). If an identifier follows the end that matches the PROC-ident, the compiler will issue a error if its not the same. This gives the programmer an opportunity of localizing mismatched ends.

7. call -> PROC-var

Procedures may be invoked using the call statement. The variable identifier PROC-var is sematically restricted to built-in functions, runtime support procedures or a PROC_ident declared in a procedure statement.

If there are actual parameters appearing with the PROC-var, their number may not exceed the number of formal parameters declared in the procedure statement. It may, however, be less than the number declared. In this case no assignment is made and the value of the formal parameter will be unchanged from the previous call. Parameters are passed by value or reference depending on the formal parameter declaration.

Only word addresses are passed to call by reference parameters. If address truncation is necessary, a warning is output (e.g. passing by reference the second element of a halfword or byte array). If the parameter is an expression, a temporary location is generated, and its address is passed and a warning is output. If a constant is passed by reference, a warning is output since modification of the parameter by the called routine would change the constant for the rest of the program execution.

fortran procedure parameters are passed by reference only.

The storage management algorithm used XPL/S programs does not support recursive procedure calls.

8. return -> return [exp]

When it appears in a procedure, the return statement causes a return branch to the instruction following the call. If it appears in a main program, a normal return to the operating system is generated.

If the optional expression appears, then it is taken as the value of the procedure. If the procedure has character type, and exp is numeric, automatic conversion to a string is performed.

9. exit -> exit [LABEL-ident]

An exit statement causes a branch to the instruction following the end of the enclosing group. If a label identifier appears, then the target of the branch is the end that matches the do that bears the the label. The label must be on a group that encloses the exit statement.

For example:

```
E DO;
   DO;
      EXIT;
      EXIT E;
   END;  /*  TARGET OF FIRST EXIT  */
END;  /*  TARGET OF EXIT E  */
```

Note that an exit statement has no meaning if it appears outside a group.

10. assgt -> (, . var)+ = exp

An assignment statement consists of one or more variables, separated by commas, to which the value of the exp on the right hand side of the assignment operator, '=', is assigned. If the var is of character type and exp is numeric then automatic conversion is performed. Assignment of character exp to numberic var is permitted but no conversion takes place. The result is to assign the descriptor of the exp to the var.

If character and numeric var's are mixed on the left of a numeric exp, conversion takes place on the appearance of the character var nearest to the assignment operator, i.e., moving from right to left.

Procedure names (PROC-vars) are permitted on the left hand side of an assignment. In this case the exp is evaluated and passed to the procedure as an implicit actual parameter, following the explicit actual parameters, if any. It should be noted, however, that the other actual parameters to the procedure are evaluated and stored before exp is evaluated. Thus if the same procedure name apears on both sides of the assignment, or more than once on the left, the explicit parameters of the rightmost call will override the others.

If a stack variable appears on the left of an assignment operator, the value of the exp is pushed onto the stack.

There is no provision made for informing a procedure if it is called on the left; the programmer must devise his own, as in the

following example:

```
PROCEDURE FILE(UNIT, KEY, BUF) CHARACTER;
   DECLARE(UNIT, KEY) FIXED, BUF CHARACTER INITIAL('');
   PREC(UNIT, POS(KEY));
   IF LENGTH(BUF) = 0 THEN /* ON RIGHT */
      RETURN INPUT(UNIT);
   /* ON LEFT */
   OUTPUT(UNIT) = BUF; BUF = '';
END FILE;

RCD = FILE('M:BI', 3);
FILE('M:BI', 5) = RCD;
FILE(A,B) = FILE(B,C);
```

The first two examples are valid, the third is not; its result would be to write on unit B, key C. The scheme requires that length zero records not be written.

```
11.  decl -> declare { , . { ident literally string
                           | {{ ident | ( { , . ident }+ ) }
                                [ (CONST-exp) ]
                             { fixed | stack | character
                             | bit(CONST-exp) | floating }
                             [global] initial ({,.  CONST-exp]}+)
                             | ref
                             | external ]}}}
```

A declaration consists of the reserved word declare followed by one or more declaration elements, separated by commas. These elements may consist of macro declarations and/or type declarations. The scope of a declaration is the procedure in which it appears. An identifier may not be re-declared unless the declaration is made inside a procedure nested within the scope of the original declaration.

ident literally string

XPL/S provides a simple text-replacement macro facility. When the lexical analyzer encounters ident, it is replaced by the specified string and re-scanned. This text may contain other macro identifiers, but not, of course, the original ident. Note that the string quotes are part of the declaration and are not included in the replacement text.

There is a modified scope for macros; once the declaration is made, it remains in force until the scope is closed. In other words, one may not re-declare a macro identifier.

Macros are particularly useful for the naming of constants: this practice permits all occurrences to be changed at once and improves the readability of the code.

Example:

```
DECLARE TRUE LITERALLY '1',
   FOREVER LITERALLY 'WHILE TRUE';
DO FOREVER;
```

FOREVER expands to: WHILE 1.

The type declaration may be considered in three parts: the specification of name and array bound, the specification of type and the specification of initial value.

```
{ ident | ( ( , . ident )+ ) } [ (CONST-exp) ]
```

This alternative for declaration element defines typed variables. A factored declaration consists of a series of identifiers enclosed in parentheses and separated by commas: ((, . ident)+). Thus, a type declaration may begin either with 'ident' or '(ident1, ident2, ... ,identN)'. If the declaration is factored, all the idents will have the same attributes.

Array bounds are optional. They have the form: (CONST-exp). The CONST-exp is an expression that evaluates to a numeric constant at compile-time. Only one dimension is permitted, and, as zero-origin indexing is used for subscripted variables. Array bounds are ignored for formal parameters passed by reference and for data declared external and hence is useful only for documentation.

```
DECLARE X(5) FIXED;
```

reserves 6 locations.

There are five types: fixed | stack | character | bit (CONST-exp) | floating.

fixed type corresponds to 32 bit signed integers on the Sigma. Operations are compiled using fullword logic. Use of fixed variables tends to minimize the amount of code generated, as certain operators, such as xor, |, etc. do not have halfword or byte equivalents.

floating type is treated as fixed.

stack variables allow access to the Sigma hardware stack. The number of cells allocated from the stack is determined by the array bound; hence, there is no "array of stacks". If the stack is not initialized, the appropriate stack pointer double word is provided. For example,

```
DECLARE S(5) STACK;
```

requires eight words of storage. Stacks begin on a doubleword boundary. The first two words are the stack pointer. An even number of words is always allocated, if necessary one more than the declared array bound. A stack variable is accessed like any other. Appearance of the identifier causes a push or pull to be generated, according to the context.

character type variables proved the XPL/S string facility. Each character variable has two parts, the descriptor and the string. Descriptors are allocated in a fullword of memory. The first byte (bits 0-7) contains the length of the string. A length of zero signifies the null string (''). The remaining three bytes (bits 8-31) contain the absolute byte resolution address of the first character. Character variables are dynamic: both length and address parts may change from time to time. The use of one byte for the length field places a limitation of 255 characters on the length of a string.

bit variables allow access to the other addressing modes on the Sigma, as well as providing the opportunity for some documentation as to the range of variables. The CONST-exp determines the storage type used; note the distinction between short and long bit strings.

CONST-exp	storage	
1 - 8	byte	
9 - 16	halfword	short bit strings
17 - 32	fullword	
33 - 2040	character	long bit strings

Long bit strings, bit variables of length > 32, are treated as character variables, assigned a descriptor and restricted to character operations.

Before making use of byte or halfword variables, the programmer should familiarize himself with the peculiarities of byte and halfword operations on the Sigma. Two of the pitfalls are illustrated below.

```
DECLARE (H1,H2) BIT(16) INITIAL("FFFF", "FFFF"),
    B BIT(8)  INITIAL ("1FF");
```

H1=H2 is true, H1="FFFF" is not. This is due to the fact that a halfword load and compare H1=H2 involves sign extension while H1="FFFF" uses a compare immediate, which does not. B="FF" is true, B="1FF" is not, due to the truncation that takes place with byte variables and the fact that there is no sign extension.

Variables declared global in one compilation may be declared external in another with linkage provided by the loader.

Initializations are specified by: initial ([, . CONST-exp]+) external variables and formal parameters declared ref may not be initialized.

This construct permits the compile-time initialization of storage. The CONST-exps are evaluated and assigned to successive storage locations, beginning with the first and continuing to the array bound, if specified.

Usually, numeric expressions will be used for numeric variables and strings will be used for character variables. However, there are some applications for unusual initializations. Note that the conversions described here are inapplicable elsewhere in the language.

If the storage type is 32 bits or less, and a character string

appears as initialization, the appropriate number of characters (1,2, or 4) are used from the left end of the string. Characters are right justified with zero fill. If a character or long bit string is being initialized there are two cases to consider. If the CONST-exp is a decimal constant or an expression, then it is converted to a character string. If a bit string (long or short) appears, then it is used according to the rules for long bit string constants described in the section on lexical entities. For example,

```
DECLARE CR CHARACTER INITIAL ("0D");
```

may be used to obtain a carriage return character in a form suitable for concatenation.

Factored declarations may be initialized. Storage is initialized in contiguous elements, from left to right, including cells that would otherwise be ignored, that is, in the case of byte or halfword arrays that do not require an integral number of words. All declarations begin on a word boundary.

Stack variables may be initialized. Note, however, that if this is done, the programmer must also provide the stack pointer doubleword, as in the following example.

```
DECLARE S(5) STACK INITIAL(ADDR(S(3)), "80048002",1,2);
```

Default initialization is 0 for fixed variables and short bit strings and '' (null string) for character and long bit string variables.

Examples:

```
DECLARE A(5) BIT(7) INITIAL('1',1,'21');
```

generates the following bytes, beginning on a word boundary.

```
F1  01  F2  F1   00  00  X  X
```

where cells marked 'X' are not used.

```
DECLARE C(3) CHARACTER INITIAL('ABC', 123, "123");
```

generates descriptors (length/address) assuming a base of 1C00:

```
3/1C00   3/1C03   2/1C06   0/0
```

and strings:

```
C1C2C3F1F2F31230.
```

```
DECLARE X FIXED INITIAL ('X');
```

generates

```
000000E7
```

ref specifies that a formal parameter is called by reference. The array bound, if any, is ignored and hence is only useful for documentation. Only formal parameters may be of type ref and they may not be initialized. Data may be passed by reference within the same compilation.

external specifies that the data has been allocated by another module. Appropriate code is generated using an external reference which must be satisfied at loadtime. No storage is allocated in the module being compiled. Array bounds, if specified are ignored and are useful only for documentation.

12. exp -> { | | xor | & . [¬] log-prim }+

This production defines the highest priority operators. Complementation is performed with an exclusive-or operator. That is, ¬A = A xor "FFFFFFFF".

13. log-prim -> {{{ = | > | < | >= | <= | ¬= | ¬> | ¬< } . str-exp }+
| overflow | underflow }

The first part of this production defines a logical primary as a string expression, or as two string expressions separated by a relational operator. Normally, both operands are of the same type, i.e., either both numeric or both string. If the operands are numeric, the relation is established using an appropriate compare instruction. If they are strings, the comparison is performed on the following algorithm.

Let a and b be strings. Then the value of a ø b, where ø is a relational operator, is:

1. if length(a) ¬= length(b) then length(a) ø length(b)
2. if length(a) = length(b), then the strings are compared on a character by character basis. If a and b first differ in the i-th character, the result is BYTE(a,i) ø BYTE(b,i). As this implies, the comparison treats the characters as integers; collating sequence is not used.

If one operand is a string and the other is numeric, the numeric is converted to its decimal equivalent and a string comparison is performed. This conversion is relatively expensive, so care should be taken not to cause such conversions to be made accidently.

underflow and overflow are 0-ary operators that generate the appropriate tests for stack operations. They must follow the stack operation immediately. For example,

```
DECLARE S(10) STACK, X FIXED;
S=X;  IF OVERFLOW THEN...;
X=S;  IF UNDERFLOW THEN...;
```

Of course, the stack pointer doubleword must not be initialized in such a way as to force a trap on these conditions.

As is implied, use of the ~, &, | and xor operators with underflow and overflow is permitted.

14. str-exp -> { str-exp || . arith-exp }+

The || operator causes the concatenation of strings. If either of the operands is not a string, automatic conversion to EBCDIC, is invoked.

Concatenation causes a new string to be created in the free string area, even if one of the operands is the null string (''). If both operands are '', no action is taken. If the left-hand operand is the last string in the free sring area, only the right-hand operand is copied.

15. arith-exp -> { + | - | * | / | mod . [+ | -] arith-primary }+

These operations are normally performed upon numeric data types, and have the usual meanings.

16. arith-primary -> var | (arith-exp) | number | string

There are no situations in the evaluation of expressions in which an automatic conversion from character type to numeric type (such as fixed) takes place. Conversion from numeric to character takes place automatically in the situations: (1) When a numeric value is assigned to a character or long bit string variable. (2) When a numeric expression appears as one of the operands of the || operator. (3) When a numeric value is an actual parameter to a procedure whose formal parameter is a character or long bit string variable.

17. var -> ident [(, [.exp])]

A variable is an identifier, optionally with a subscript or argument list. A subscript is a single expression enclosed in parentheses. Only one subscript is permitted. Identifiers need not be declared with a dimension to be subscripted. A stack variable is pulled when it appears in an expression and pushed when it appears on the left of an assignment operator. Note that each occurance of the identifier generates one push or pull. The effective address of a subscripted variable is computed zero-origin, without regard for array bounds. For example, given the declaration:

```
DECLARE A BIT(8),
        B FIXED,
        C (3) FIXED,
        D FIXED;
```

A(4) is the first byte of B, C(-1) is B, C(4) is D, etc. Whenever a subscript is a CONST-exp that resolves to a word boundary, that address is used in generating instructions; otherwise subscripted

instructions are required.

XPL/S Operators

Symbol	Priority Class	Interpretation
|	7	logical inclusive or, 32 bits
xor		logical exclusive or, 32 bits
&	6	logical and, 32 bits
¬	5	logical complement, 32 bits
=	4	relational equality
¬=		inequality
>		greater
>=		greater or equal
<		less
<=		less or equal
¬>		not greater
¬<		not less
||	3	concatenation of strings
+	2	addition (unary: no-op)
-		subtraction (unary: 2's complement)
*	1	product, low-order 32 bits
/		integer dividend
mod		integer remainder

3. Lexical Entities

The syntactic description refers to three terminal symbols that require further elaboration: identifier, number and string. These may be defined syntatically in terms of the more familiar symbols, letters and digits. However, this description can become quite involved and it is more usual to adopt an informal approach that is, hopefully, more informative. In addition, there is one entity not mentioned in the grammar, the comment.

Identifiers

These are the sequences of characters that may be used to represent identifiers in XPL/3. The limitations on such sequences is that they must begin with a letter and be no more than 255 characters in length. Identifiers are terminated by the appearance of one of the non-alphanumeric characters that appear in the grammar, such as ';' or a blank.

Digits are: 0,1,...,9. Letters are the upper case alphabetic characters, A,B,...,Z, plus the 9 'break' characters: _ $ @ # ` . ¢ ? and : .

No identifier may be a reserved word, that is, one of the words that are underlined in the grammar. These are as follows.

bit	else	fortran	otherwise	then
by	exit	global	overflow	to
case	external	if	procedure	underflow
character	end	initial	ref	until
do	fixed	literally	return	while
declare	floating	mod	stack	xor

Generally, inappropriate use of any of the above will cause syntax errors.

Identifiers must be declared before use. This means that they must appear either in a declaration statement (production 11) or as a procedure identifier in a procedure declaration (production 6) or on a group (production 5).

In addition to the reserved words, there two other classes of identifiers that have a special significance, the built-in functions and runtime support procedures. The built-in functions are recognized by the compiler, and code is generated to perform their defined functions. However they are not reserved words and may be redeclared. When this happens, the new declaration overrides the special meaning for the scope of the declaration. The runtime support procedures are compiler declared external procedures and are called like any other procedure.

Numbers

Numbers, or numeric constants, come in two forms: decimal constants and short bit string constants. Basically, a number is something that will fit into 32 or less bits of storage. Numeric

constants that require less than 32 bits are stored right justified with leading zeros.

Decimal constants

Decimal constants are defined as a sequence of the digits, 0,1,...9. There is no limitation as to the number of digits, however, if the resultant value is too large to be stored in a 32-bit integer, truncation will result. Note that no sign (+|-) is included in the definition. Signed constants are CONST-exps.

Short bit string constants

Bit string constants are distinguished by the enclosing double quotes. The optional term in parentheses determines the width of the digits. That is, if the width is 1, the bit string is of binary digits. The legal values are:

type	width	digits
binary	1	0,1
quartal	2	0,1,2,3
octal	3	0,1,...,7
hexadecimal	4	0,1,...,F

The default width is 4. The following are equivalent representations of the number 255.

```
*(1) 1111 1111*
*(2) 3 3  3 3 *

*(3) 3 7 7 *
*(4) F F *
* F F *
*(2) 33 (1) 1 (3) 7 *
```

The width may be changed within the bit string quotes, as illustrated by the last example.

Strings

Strings are entities that are accessed via a string descriptor. String constants are distinguished by the enclosing string quotes: single (') for character strings and double (") for long bit strings. The difference between the two is the way in which the characters are treated when they are scanned.

Character string constants

The characters permitted in a string constant are any characters, printing or non-printing, that the operating system will accept. In general, this means all the 8-bit integers from 0 to 255. The null string is represented by two adjacent quotes, ''. If the quote character is to appear in the string, it must be preceded by another quote, such as in: ''' TWAS BRILLIG', which prints as 'TWAS BRILLIG.

Long bit string constants

A long bit string constant is one that requires more than 32 bits of storage. In appearance, it resembles a short bit string. However, rather than being stored in a single word, its digits are assembled into bytes and stored as a character string. If the long bit string does not fit into an integral number of 8-bit bytes, then it is stored left adjusted with zero fill at the right of the last byte. The following example shows the external and internal representation of the hexadecimal digits; first as a character string and second as a long bit string.

```
'0123456789ABCDEF'      F0F1F2F3F4F5F6F7F8F9C1C2C3C4C5C6
"0123456789ABCDEF"      0123456789ABCDEF
```

Long bit strings are regarded as character strings, they may only be compared and concatenated.

Comments

XPL/S permits arbitrary text to be enclosed within the "comment brackets" '/*' and '*/'. This text, which may extend over as many cards as desired, is ignored by the compiler, with one exception. If the '$' character appears, then the following characters are interpreted as control options. This interpretation ceases at the end of the source record, comment, or at the appearance of a period. In the following example, there is no confusion over the various uses of the characters "LO" as a control option and a variable name.

```
/* $LO */ LO = LO + 1;
/* $^LO.  LO OFF */
```

For the control options and their meaning, see the XPL/S Technical Note.

4. Built-in Functions

ABEND	ASHR	BYTE	COREWORD	ENDOF	SHL
ABS	BIT8	CHAR	CSHL	LENGTH	SHR
ADDR	BIT16	COREBYTE	CSHR	NEW	SUBSTR
ASHL	BIT32	COREHALFWORD	DESC	RTN-ADDR	

The built-in functions are names which are recognized by the compiler and have predefined significance. They are not reserved words and may be re-declared, which destroys the original meaning within the scope of the new declaration. The functions are implemented by inline code.

ABEND

When called, ABEND generates an abnormal exit to the monitor.

```
ABEND;
```

becomes

```
CAL1,9  3
```

ABS

Returns absolute value of its argument.

ADDR

Delivers the word-resolution address of its argument. Valid arguments are fixed or character variables, subscripted or not, constants, procedures, labels and external routines. Invalid arguments are expressions, subscripted bit variables with size less than 17, or constant subscripted bit variables where the subscript does not evaluate to a word boundary. N.B.: ADDR itself is a constant in items requiring a CONST-exp, but not if it appears in an expression with other terms, or if its argument has non-constant components.

A second, optional, parameter is permitted. If present, it specifies the resolution of the address to be formed. Legal values are:

0: Byte resolution
1: Halfword resolution
2: Word resolution
3: Doubleword resolution

The default is word resolution. Note that the resolution parameter does not alter the requirement that the first parameter have a word boundary address. Some examples of use follow.

```
ADDR(I)     word
ADDR(I,0)   byte
ADDR(I,1)   halfword
ADDR(I,2)   word
ADDR(I,3)   doubleword
```

ASHL

Analogous to SHL except that an arithmetic shift left is generated.

ASHR

Analogous to SHR, except that an arithmetic shift right is generated. The sign bit is propagated right in the shift.

BIT8

Performs type transfer on argument to bit(8). This allows, for instance, a fixed variable to be treated as a bit(8) array.

```
X = BIT8(A(I));
```

assigns to X the I-th byte of the aray.

BIT16

Performs type transfer on argument to bit(16).

```
X = BIT16(A(3));
```

assigns to X the 3rd halfword of the array.

BIT32

Performs type transfer to bit(32). Allows, for instance, a bit(8) array to be moved using fullword instruction.

```
X = BIT32(A(I));
```

assigns to X the I-th word of the array.

BYTE

This function permits strings to be treated as an array of characters, with 0-indexing. BYTE may appear on either side of an assignment. Two forms are allowed:

```
BYTE (CHAR-var)
BYTE (CHAR-var, exp)
```

The first form is the same as the second, but with exp assumed to be zero. In both cases, the expth character of CHAR-var is either fetched or stored, depending on the context. The characters fetched or stored by BYTE are treated as integers. Optional index validity checking is available with the SB option. See the XPL/S Technical Note.

```
BYTE ('123') = "F0";
```

changes '123' to '023',

```
X = BYTE ('123',1);
```

sets X to "F2".

CHAR

The CHAR function performs fixed to character type transfer. CHAR is used to assign a descriptor to a character variable.

Two forms of CHAR are allowed.

```
CHAR(descriptor)
CHAR(length, address)
```

The first form expects a full 32-bit descriptor with length in bits 24-31 and absolute byte address of the first character of the string in bits 0-23. The second form expects a length (0-255) as the first parameter and an absolute byte address as the second parameter.

COREBYTE, COREHALFWORD, COREWORD

Allows all of memory to be accessed as a byte, halfword or word array, (eg. COREBYTE(N) accesses byte N in memory).

CSHL

Analogous to SHL except that a circular shift left is generated. Bits shifted off at the left are shifted on at the right.

CSHR

Analogous to SHR except that a circular shift right is generated. Bits shifted off at the right are shifted on at the left.

DESC

The DESC function performs character to fixed type transfer. It delivers the descriptor of the character variable that is its argument as a fixed quantity.

```
DESC(character-variable)
```

ENDOF

Returns on index to the end of the character variable that is its argument. Suitable for string indexing using BYTE, as ENDOF(S) = LENGTH(S) - 1.

LENGTH

Delivers the length of the string that is its argument.

```
X = LENGTH ('123');
```

sets X to 3.

NEW

Provides a new copy of its argument character variable. This is accomplished by copying into the free string area. The NEW function simulates an assign of string as well as descriptor, whereas the normal assign of a character variable assigns only the descriptor. If an external character variable is assigned to a constant string in an XPL/S compilation that is part of an overlay, NEW must be used in the assignment so that the string will remain when a new overlay is brought in on top of the old.

RTN_ADR

Provides the runtime return address to the calling procedure.

SHL

Generates a left shift logical single on the value of the first argument by the number of bits specified by the second argument, which may be a constant or a variable. For example

```
X = SHL (X,4);
```

multiplies X by 16. If the second argument is negative, the shift will be to the right. As the shift is logical, bits shifted off the left end are lost, bits shifted in at the right end are zero.

SHR

Generates a right shift logical single in the value of the first argument by the number of bits specified by the second argument, which may be a constant or a variable. If it is negative, a shift left results. As the shift is logical bits shifted off the right are lost and zero bits are shifted in at left.

```
X = SHR (X,I)
```

divides X by 2 to the I-th power if I > 0 and multiplies X by 2 to the I-th power if I < 0.

SUBSTR

A function for computing substrings. It has two forms:

```
SUBSTR (CHAR-var, exp)
```

computes the substring of the character variable, CHAR-var, beginning at the exp-th character to the end of the string.

```
SUBSTR (CHAR-var, exp1, exp2)
```

computes the substring of CHAR-var beginning at the exp-th character and of length exp2. Note that 0-indexing is used; that is, the characters are numbered 0,1,... .

```
SUBSTR ('ABCD',2) = 'CD'
SUBSTR ('ABCD'0,2) = 'AB'
```

The SUBSTR operation is performed on the string descriptors, so no new strings are created. The descriptor created points to the original characters in memory.

When computing substrings, an optional check is available to insure that the boundaries of the source string, CHAR-var, are not exceeded. Errors may occur in the position or length fields. For this, use the SB option, documented in the XPL/S Technical Note. One common error is the computation of negative length, which results in a very long string, due to the peculiarities of byte arithmetic.

5. Runtime Support

These compiler declared external procedures are part of the runtime support that is loaded with every XPL/S program. The formal parameter FIX is of type FIXED, FIX_REF is FIXED REF, FIX_CHR is FIXED but indicates that a fixed DCB address or character DCB name may be used and CHR is of type CHARACTER. Functions have their returned type indicated. The procedure names are not reserved and may be redeclared within the usual rules for scope.

```
BUFFERIO  (FIX_CHR);
```

Causes subsequent input or output on the designated DCB to be buffered. (Supported under RBM only)

```
BUFFERIO('M:SI');
```

```
CTL_CARD  CHARACTER;
```

Returns the card image of the control card used to invoke the program. (Supported under RBM only)

```
CARD = CTL_CARD;
```

```
DEC  (FIX) CHARACTER;
```

This function performs binary to decimal conversion in its argument.

```
X = 1234;
STR = DEC (X);
```

stores the string '1234' in the character variable STR. DEC is used by the compiler to perform number-to-string conversion. It may also be used explicitly for the same purpose, where such conversion is not automatic.

```
L = LENGTH(DEC(1234));
```

```
GET_DCB  (CHR) FIXED;
```

Returns the address of the specified DCB. This address may then be used as a parameter to the input/output routines.

```
DCB = GET_DCB('M:BO');
```

The above example fetches the address of the M:BO DCB.

```
HEX  (FIX) CHARACTER;
```

Converts its argument to a hexadecimal string.

```
STR = HEX(X);
```

```
INPUT  (FIX_CHR) CHARACTER;
```

Character input from designated DCB. String input is obtained using the INPUT function, which delivers either the next record from the device assigned to the designated DCB or the null string if an eof or eod is sensed on the device. Aside from these two conditions, and "lost data", which is ignored, all other errors result in program termination. The argument to INPUT is either th DCB name or DCB address. If no argument is given, M:SI is assumed. DCB addresses may be obtained using the external function GET_DCB from the runtime support. INPUT is slightly faster if an address is used. Buffering may be requested by using BUFFERIO in the runtime support. (DCBs other than M:SI must be assigned by the user.) The following two examples are equivalent.

```
S = INPUT;
S = INPUT('M:SI');
```

```
INPUTA  (FIX_CHR, FIX_REF) FIXED;
```

Binary (array) input from DCB designated by first parameter to buffer whose address is the second parameter. Note that the DCB RECL parameter determines the record size, not the declared size of the buffer. Actual record size is returned through the entry point.

```
LEN = INPUTA('M:BI', BUFFER);
```

```
INPUTAK  (FIX_CHR, FIX_REF, FIX);
```

The same as INPUTA, with the addition of the key parameter. (Not supported under RBM)

```
INPUTAK(DCB, BUF, KEY);
```

```
INPUTK  (FIX_CHR, FIX);
```

The same as INPUT, with the addition of the key parameter, except that it must be declared external and has no default DCB parameter. As keyed files may have zero-length record (meaning all blank) INPUTK will return a single blank (' ') for such a record in order that it may be distinguished from end-of-file. (Not supported under RBM)

```
STR = INPUTK(DCB, KEY);
```



LOAD_SEGMENT (FIX);

Loads an overlay segment and removes all descriptor areas higher in core or overlapping the first word address of the segment, from the liat of active descriptor areas that is used in free string area garbage collection. (See Appendix G.)

OUTPUT (FIX_CHR);

Provides for string output to external devices, assigned to the DCB whose name appears as the argument. If no argument is specified, M:LO is assumed. The M:LO DCB uses Vertical Format Control. That is, the first character of each record is not printed. As with INPUT, the argument may be either a DCB name or a DCB address and buffering may be requested by a call to BUFFERIO. DCB's other than M:LO must be assigned by the user. The following two examples are equivalent.

```
OUTPUT = S;
OUTPUT ('M:LO') = S;
```

OUTPUT does not pad strings to the record size of the file, but instead writes the actual number of characters in the string. On devices to be used subsequently for INPUT, it is advisable to pad strings to the record length of the file. Empty output ('') is equivalent to output of a single space, i.e. OUTPUT = '' has line same effect as OUTPUT = ' '.

OUTPUTA (FIX_CHR, FIX_REF);

Binary (array) output to DCB designated by first parameter from buffer whose address is the second parameter. Note that RECL of the DCB determines record size, not declared size of the buffer.

```
OUTPUTA('M:BO', BUFFER);
```

OUTPUTAK (FIX_CHR, FIX_REF, FIX);

The same as OUTPUTA, with the addition of the key parameter. (Not supported under RBM)

```
OUTPUTAK(DCB, BUF, KEY);
```

OUTPUTK (FIX_CHR, FIX, FIX);

The same as OUTPUT, with the addition of the key parameter, except that it must be declared external and has no default DCB parameter. (Not supported under RBM)

```
OUTPUTK(DCB, KEY) = 'XXX';
```

```
PFILEB  (FIX_CHR);
```

Position file backward. Position after operation is before (on the forward sense) of previous end-of-file mark on tape or at the beginning of a RAD file. Equivalent to M:PFIL,BOF

```
        PFILEB('M:CI');
```

```
PFILEF  (FIX_CHR);
```

Position file forward. Position after operation is before next end-of-file mark on tape or at the end of a RAD file. Equivalent to M:PFIL,EOF

```
        PFILEF('M:BO');
```

```
PREC  (FIX_CHR, FIX);
```

Position record on RAD or tape. First argument is DCB, second is number of records to move. If this is negative, positioning is backward. Operation stops on end-of-file. Note: the PREC operation is very slow on compressed format files.

```
        PREC('M:CO', 3);
        PREC(DCB, -NREC);
```

```
REWIND  (FIX_CHR);
```

File manipulation, similar to the !REWIND card, except that program does not wait for completion. Rather, the next access is queued until rewind is complete.

```
        REWIND('M:BI');
```

```
SFILEB  (FIX_CHR);
```

Skip file backward. Position after operation is after (in the backward sense) the previous end-of-file mark on tape, or at the beginning of the tape, if in first file. Do not use on a RAD file. (Supported under RBM only)

```
        SFILEB('M:BO');
```

```
SFILEF  (FIX_CHR);
```

Skip file forward. Position after operation is after next end-of-file mark on tape. Do not use on a RAD file. (Supported under RBM only)

```
        SFILEF('M:BO');
```

TEST_EOF (FIX_CHR);

Returns true (1) if the last operation on the designed file resulted in end-of-file data (error code "05") or end-of-file (error code "06").

```
CONDITION = TEST_EOF('M:BI');
```

TIME CHARACTER;

Returns the character string representation of the system date and time.

UNLOAD (FIX_CHR);

File manipulation, similar to !UNLOAD. (Supported under RBM only)

```
UNLOAD('M:BO');
```

WEOF (FIX_CHR);

Writes a single end-of-file mark on the device or file assigned to the specified DCB.

```
WEOF('M:BO');
```

In addition a compile time option, SB, will generate code to detect BYTE or SUBSTR operations that would result in an index outside the string or an invalid substring. Overhead generated is 3 words per BYTE and 5 words per SUBSTR plus the call. The length of concatenated strings is checked for overflow.

Appendix A Compiler Messages

The compile time messages are errors, warnings and other messages in response to compile time options (eg. FR). Error messages identify syntactic or semantic problems while warnings are used to identify potential semantic errors that may require correction (eg. passing an expression by reference). Each is prefixed by ERROR: or WARNING:.

ERRORS:

ARITHMETIC WITH STRINGS
One or both operands of &, |, ^, +, -, *, /, mod or xor is a string. Operation is compiled using string descriptor as operand.

ASSIGNMENT NEEDS ILLEGAL TYPE CONVERSION
Non-constant initialization; built-in function other than BYTE on left of assignment.

CONSTANT TABLE OVERFLOW
Too many unique numeric constants have been detected by the redundant constant algorithm. Program correctness is unaffected. Note: does not include constants appearing in declaration initializations.

CORRECT REGISTER TYPE UNAVAILABLE
Excessively complex expression has resulted in the use of all of the required type of register.

DUPLICATE DECLARATION FOR name
Name has already been declared within the current scope.

EXPRESSION MUST EVALUATE TO A CONSTANT
Non-constant (i.e. variable) term appears in a CONST-exp.

EXIT OUTSIDE GROUP
EXIT has no meaning outside a group. Statement is ignored.

FORTRAN PARAM name SHOULD BE FIXED,REF
In a fortran procedure, a formal parmeter has been declared something other than fixed ref, the default if there is no declaration.

GLOBAL PROCEDURE name NOT ON LEVEL ZERO
global procedure name definition nested inside an enclosing procedure.

<EXPRESSION> IS OF TYPE CHARACTER BUT PROCEDURE IS NOT
The value of a return expression is of type character, but the enclosing procedure is not.

ILLEGAL BIT STRING WIDTH: n
The construct "(n) ..." with n < 1 or n > 4.

ILLEGAL CHARACTER: "xx"
A character whose hexadecimal value is xx has been scanned in some context where a legal XPL/3 character is expected.

ILLEGAL CHARACTER IN BIT STRING: n
The character n is not one of the legal bit string digits, as defined by the current width.

ILLEGAL RESOLUTION
If first argument to the two-argument form of ADDR may be only 0, 1, 2 or 3.

ILLEGAL SYMBOL PAIR <x> <y>
Top of parse stack and next symbol are incompatible.

ILLEGAL USE OF function
Builtin function used incorrectly.

ITERATION VARIABLE name SHOULD NOT BE REF
Iteration variable name is declared ref.

LABEL ON END DOES NOT MATCH name ON DO
Indicates the mismatching of DO...END pairs. Name, the LABEL-ident on the matching DO, was expected at this point. If name is blank, no label was expected.

LABEL ON END DOES NOT MATCH name ON PROCEDURE
Indicates the mismatching of procedure name and the corresponding end. Name, the PROC-ident of the procedure currently being compiled, was expected.

MACRO EXPANSION TOO LONG
Macro has expanded to more than 255 characters.

MACRO TABLE OVERFLOW
Too many literally declarations.

MISSING) IN BIT STRING
The bit string width is not followed by a); some other nonblank character has been scanned.

MULTIPLE SUBSCRIPTS NOT ALLOWED
More than one subscript.

name IS NOT A PROCEDURE
name has appeared in a call statement, but is not a procedure.

name NOT A FORMAL PARAMETER
name is declared ref but is not a formal parameter.

NEED CHARACTER TYPE
Argument to BYTE, SUBSTR, ENDOF or LENGTH is not a character string or character variable.

NOT A WORD BOUNDARY ADDRESS
The parameter of ADDR or a REF parameter of a procedure cannot be resolved to a compiletime word boundary address (e.g., it is an expression).

ONLY FORTRAN PROCEDURE MAY HAVE UNDECLARED PARAMETERS
external procedure declaration has formal parameters but no declaration statements.

OTHERWISE NEEDS CASE
Otherwise ending appear on other than case group.

PARSE STACK OVERFLOW
Excessive nesting or error resulted in excessive parse stack size. Compilation stops at this point.

PROCEDURE CALLS ONLY IN SIMPLE ASSIGNMENTS
If a procedure identifier appears on the left of an assignment, it may not be followed by a comma.

PROGRAM ENDS PREMATURELY
Compiler has attempted to re-read the program end-of-file. Usually results from mismatched quotes ('or"), missing commend-end (*/) or DOs not matched by ENDs.

RESOLUTION PARAMETER NOT ALLOWED ON ADDR OF REF VARIABLE
A resolution parmeter has been specified for the ADDR built-in function when variable is of type ref.

"RETURN <EXPRESSION>" NOT ALLOWED FOR UNTYPED PROCEDURE
Only simple return statements are allowed in a procedure that has no declared type.

SHIFT REQUIRES TWO ARGUMENTS
SHL, SHR, ASHL, ASHR, CSHL AND CSHR require an operand and a shift count.

STRING NOT ALLOWED AS A SUBSCRIPT
Construct such as A('I') has been detected.

STRING TOO LONG
More than 255 characters in a string. Check for mismatched quotes.

SUBSCRIPTED DO VARIABLE
Do variable subscripted. E.G. DO K(J) = 1 TO 8;

SYMBOL TABLE OVERFLOW
Too many names have been entered in symbol table. Compilation ceases at this point.

TARGET OF EXIT NOT A LABEL ON ENCLOSING GROUP

TOO MANY EXPANSIONS
The current macro has caused too many nested expansions. Probably the macro is recursive, as in DECLARE X LITERALLY 'X';

TOO MANY ITEMS IN INITIAL LIST
Initial list longer than declared size of array.

TOO MANY PARAMETERS
The number of actual parameters to a procedure exceeds the number of formal parameters.

UNDECLARED IDENTIFIER name
Name has not been declared. Default declaration with type of fixed is made.

WRONG NUMBER OF ARGUMENTS
Built-in function has too many or too few arguments.

WARNING:

TEMPORARY GENERATED FOR REF
It has been necessary to allocate a storage location to temporarily store a value so that it may be passed by ref. (eg. an expression such as I+2 in a fortran procedure call).

TRUNCATED TO WORD ADDRESS
An address as been truncated to a word boundry (eg. passing B(1) by reference when B is declared BIT(8)).

WORD ADDRESS OF A CONSTANT
The address of a constant is passed by reference. If the called procedure changes the value of its parameter, the constant is changed for the rest of the program execution.

Other Messages

PARTIAL PARSE TO THIS POINT IS:
The stack (bottom on left, top on right) is followed by ellipses (...) and the current token, that is, the item that the syntax analyzer is attempting to process, a pointer (->) and the lookahead symbol.

REDUCTION: n
The PR, follow reductions, option is on and reduction n has occurred.

RESUMED WITH SYMBOL "<symbol>"
Parser has restarted syntax scan after an error.

Appendix B Runtime Support Error Messages

All runtime support errors are fatal. Control is returned to the operating system via M:ERR. Location aaaa is the return address from the runtime support if the error had not occurred.

Appendix B Runtime Support Error Messages

All runtime support errors are fatal. Control is returned to the operating system via M:ERR. Location aaaa is the return address from the runtime support if the error had not occurred.

ACTIVE DESCR AREA LIST OVERFLOW AT LOC aaaa
A call to a global procedure has requested that its descriptor area be added to the list of active areas (if not already in the list) to be checked at garbage collection. Change X:ADASIZ by a :MODIFY card at load time.

CASE OUT OF RANGE AT LOC aaaa
The index of the case statement prior to aaaa was out of range and no otherwise statement was provided.

INSUFFICIENT DYNAMIC MEMORY AT LOC aaaa
Program fills available memory. As a result there is insufficient core for the required initial allocation of free string area, or allocation following a BUFFERIO request.

INSUFFICIENT STRING SPACE AT LOC aaaa
A major compactification of the string area has failed to release sufficient space.

INVALID BYTE INDEX AT LOC aaaa
With the SB option in force, BYTE index out of range occurred.

INVALID SUBSTRING AT LOC aaaa
With the SB option in force, an illegal substring was computed by SUBSTR.

I/O ERROR n ON DCB AT dcb-addr (NOT IN DCB TABLE)
Error n has occurred on the DCB at dcb-addr but the error service routine has been unable to locate that address in the program DCB table. (Does not apply to RBM)

I/O ERROR n ON FILE dcb-name AT LOC aaaa
Error n has occurred on the specified DCB. Consult the operating system reference manual for the interpretation of the error number, which is given in hexadecimal.

I/O ERROR n ON FPT AT fpt-addr AT LOC aaaa
Error n has occurred on the specified FPT. (Applies to RBM only)

NO DCB AT x FROM LOC aaaa
One of the I/O routines has been called with a DCB address (in hexadecimal) of x. This probably results from failure to initialize the DCB address variable. (Applies to RBM only)

NO DCB FOR FILE dcb-name FROM LOC aaaa
The dcb-name has been passed to an I/O routine, but no DCB by this name appears in the program's DCB table. Results from failure to include correct dcb-name on :ASSIGN or !ASSIGN card, or to REF the dcb-name.

NO SUCH SEGMENT AS N AT LOC aaaa
Argument N to LOAD_SEGMENT is not a valid segment number. (N in decimal.)

STACK OVERFLOW FROM LOC aaaa
The instruction at aaaa has caused a stack fault (underflow or overflow).

STRING TOO LONG AT LOC aaaa
A call to INPUT or a concatenation generated a string of length >255.

TOO MANY COLLECTABLE STRINGS AT LOC aaaa
The X:DXSIZE parameter is too small.

Appendix C BNF Syntax for Simple LR(1) Parser

```
1   <PROGRAM>  ::=   <STATEMENT LIST>

2   <STATEMENT LIST>  ::=   <STATEMENT>
3                         |   <STATEMENT LIST> <STATEMENT>

4   <STATEMENT>  ::=   <BASIC STATEMENT>
5                    |   <IF STATEMENT>

6   <BASIC STATEMENT>  ::=   <ASSIGNMENT> ;
7                            |   <GROUP>
8                            |   <PROCEDURE DEFINITION> ;
9                            |   <RETURN STATEMENT> ;
10                           |   <CALL STATEMENT> ;
11                           |   <EXIT STATEMENT> ;
12                           |   <DECLARATION STATEMENT> ;
13                           |   ;

14  <IF STATEMENT>  ::=   <IF CLAUSE> <STATEMENT>
15                      |   <IF CLAUSE> <TRUE PART> <STATEMENT>

16  <IF CLAUSE>  ::=   IF <EXPRESSION> THEN

17  <TRUE PART>  ::=   <BASIC STATEMENT> ELSE

18  <GROUP>  ::=   <GROUP HEAD> <GROUP ENDING>

19  <GROUP HEAD>  ::=   <GROUP NAME> <GROUP TYPE>
20                    |   <GROUP HEAD> <STATEMENT>

21  <GROUP NAME>  ::=   DO
22                    |   <IDENTIFIER> DO

23  <GROUP TYPE>  ::=   ;
24                    |   <STEP DEFINITION> ;
25                    |   <WHILE CLAUSE> ;
26                    |   <CASE SELECTOR> ;

27  <STEP DEFINITION>  ::=   <VARIABLE> = <EXPRESSION>
                               <ITERATION CONTROL>

28  <ITERATION CONTROL>  ::=   TO <EXPRESSION>
29                           |   TO <EXPRESSION> BY <EXPRESSION>

30  <WHILE CLAUSE>  ::=   WHILE <EXPRESSION>

31  <CASE SELECTOR>  ::=   CASE <EXPRESSION>

32  <PROCEDURE DEFINITION>  ::=   <PROCEDURE HEAD> <STATEMENT LIST>
                                    <ENDING>
33                              |   <EXTERNAL PROCEDURE>
```

```
34 <EXTERNAL PROCEDURE> ::= <EXTERNAL PROCEDURE HEAD> ;  <ENDING>
35                        |   <EXT WITH PARAM> ;
                               <DECLARATION LIST> <ENDING>
36                        |   <EXT WITH PARAM> ;  <ENDING>

37 <EXTERNAL PROCEDURE HEAD> ::=  <EXTERNAL PROCEDURE NAME>
38                              | <EXTERNAL PROCEDURE NAME> <TYPE>

39 <EXT WITH PARAM> ::=  <EXTERNAL PROCEDURE NAME> <PARAMETER LIST>
40                     |  <EXTERNAL PROCEDURE NAME> <PARAMETER LIST>
                           <TYPE>

41 <EXTERNAL PROCEDURE NAME>  ::=  EXTERNAL PROCEDURE <IDENTIFIER>
42                               |  FORTRAN PROCEDURE <IDENTIFIER>

43 <DECLARATION LIST> ::=  <DECLARATION STATEMENT> ;
44                      |  <DECLARATION LIST>
                           <DECLARATION STATEMENT> ;

45  <PROCEDURE HEAD>  ::=   <PROCEDURE NAME> ;
46                      |   <PROCEDURE NAME> <TYPE> ;
47                      |   <PROCEDURE NAME> <PARAMETER LIST> ;
48                      |   <PROCEDURE NAME> <PARAMETER LIST>
                              <TYPE> ;

49  <PROCEDURE NAME>  ::=   PROCEDURE <IDENTIFIER>
50                      |   GLOBAL PROCEDURE <IDENTIFIER>

51  <PARAMETER LIST>  ::=   <PARAMETER HEAD> <IDENTIFIER> )

52  <PARAMETER HEAD>  ::=   (
53                      |   <PARAMETER HEAD> <IDENTIFIER> ,

54  <ENDING>  ::=   END
55              |   END <IDENTIFIER>

56  <GROUP ENDING>  ::=   <ENDING> ;
57                      |   END UNTIL <EXPRESSION> ;
58                      |   END <OTHERWISE> <BASIC STATEMENT>

59  <OTHERWISE>  ::=   OTHERWISE

60  <RETURN STATEMENT>  ::=   RETURN
61                          |   RETURN <EXPRESSION>

62  <CALL STATEMENT>  ::=   <VARIABLE>

63  <EXIT STATEMENT>  ::=   EXIT <IDENTIFIER>
64                      |   EXIT

65  <DECLARATION STATEMENT>  ::=   DECLARE <DECLARATION ELEMENT>
66                               |   <DECLARATION STATEMENT> ,
                                     <DECLARATATION ELEMENT>

67  <DECLARATION ELEMENT>  ::=   <INITIALIZE TYPE>
68                           |   <INITIALIZE TYPE> <INITIAL LIST>
```

```
                              |   <TYPE DECLARATION> REF
69                            |   <TYPE DECLARATION> EXTERNAL
70                            |   <IDENTIFIER> LITERALLY <STRING>
71
72  <INITIALIZE TYPE> ::=     <TYPE DECLARATION>
73                          |   <TYPE DECLARATION> GLOBAL

74  <TYPE DECLARATION>  ::=     <IDENTIFIER SPECIFICATION> <TYPE>
75                            |   <BOUND HEAD> <EXPRESSION> ) <TYPE>

76  <TYPE>  ::=   FIXED
77              |   FLOATING
78              |   STACK
79              |   CHARACTER
80              |   <BIT HEAD> <EXPRESSION> )

81  <BIT HEAD>  ::=   BIT (

82  <BOUND HEAD>  ::=   <IDENTIFIER SPECIFICATION> (

83  <IDENTIFIER SPECIFICATION>  ::=   <IDENTIFIER>
84                                  |   <IDENTIFIER LIST>
                                        <IDENTIFIER> )

85  <IDENTIFIER LIST>  ::=   (
86                         |   <IDENTIFIER LIST> <IDENTIFIER> ,

87  <INITIAL LIST>  ::=   <INITIAL HEAD> <EXPRESSION> )

88  <INITIAL HEAD>  ::=   INITIAL (
89                      |   <INITIAL HEAD> <EXPRESSION> ,

90  <ASSIGNMENT>  ::=   <VARIABLE> = <EXPRESSION>
91                    |   <LEFT PART> <ASSIGNMENT>

92  <LEFT PART>  ::=   <VARIABLE> ,

93  <EXPRESSION>  ::=   <LOGICAL FACTOR>
94                    |   <EXPRESSION> | <LOGICAL FACTOR>
95                    |   <EXPRESSION> XOR <LOGICAL FACTOR>

96  <LOGICAL FACTOR>  ::=   <LOGICAL SECONDARY>
97                        |   <LOGICAL FACTOR> & <LOGICAL SECONDARY>

98  <LOGICAL SECONDARY>  ::=   <LOGICAL PRIMARY>
99                           |   ¬ <LOGICAL PRIMARY>

100 <LOGICAL PRIMARY>  ::=   <STRING EXPRESSION>
101                        |   <STRING EXPRESSION> <RELATION>
                               <STRING EXPRESSION>
102                        |   OVERFLOW
103                        |   UNDERFLOW

104 <RELATION>  ::=   =
105               |   <
106               |   >
107               |   ¬ =
```

```
108                    |   ~ <
109                    |   ~ >
110                    |   < =
111                    |   > =
112   <STRING EXPRESSION>  ::=   <ARITHMETIC EXPRESSION>
113                          |   <STRING HEAD> <ARITHMETIC EXPRESSION>
114   <STRING HEAD>  ::=   <ARITHMETIC EXPRESSION> ||
115                    |   <STRING HEAD> <ARITHMETIC EXPRESSION> ||
116   <ARITHMETIC EXPRESSION>  ::=   <TERM>
117                                |   <ARITHMETIC EXPRESSION> + <TERM>
118                                |   <ARITHMETIC EXPRESSION> - <TERM>
119                                |   + <TERM>
120                                |   - <TERM>
121   <TERM>  ::=   <PRIMARY>
122             |   <TERM> * <PRIMARY>
123             |   <TERM> / <PRIMARY>
124             |   <TERM> MOD <PRIMARY>
125   <PRIMARY>  ::=   <CONSTANT>
126                |   <VARIABLE>
127                |   ( <EXPRESSION> )
128   <VARIABLE>  ::=   <IDENTIFIER>
129                 |   <SUBSCRIPT HEAD> <EXPRESSION> )
130   <SUBSCRIPT HEAD>  ::=   <IDENTIFIER> (
131                       |   <SUBSCRIPT HEAD> <EXPRESSION> ,
132   <CONSTANT>  ::=   <STRING>
133                 |   <NUMBER>
```

Appendix D Syntactic Metalanguage

A language or a subset of a language is defined by a set of productions of the form a->b. A production is read "a is defined as b" or "a is rewritten as b". a is called the lefthand side of the production and b is called the righthand side of the production.

The basic components of the righthand side are terminal symbols and nonterminal symbols. The terminal symbols are symbols (character strings) which appear in the language (e.g. if or xor). The nonterminal symbols (also called metavariables) are symbols which are defined by appearing on the lefthand side of some production. A nonterminal symbol is any character string (usually one with some mnemonic significance) not underlined or an operator. Thus, 'exp' and 'CONST-exp' are the same metavariables: if and * are not. The 'CONST-' part means the 'exp' evaluates to a constant.

The lefthand side consists of exactly one nonterminal symbol. The righthand side consists of a sequence of terminal and nonterminal symbols interspersed with various metasymbols. The metasymbols, which include the or-metasymbol (|), the curly-bracket-metasymbols ({ }), the square-bracket-metasymbols ([ ]), and the enter-here-metasymbol (.), allow abbreviations and also make the metalanguage more readable.

The or-metasymbol allows alternatives to be defined. For example, the production letter -> A|B|C|...|Z says that the nonterminal symbol letter is defined as any one of the 26 letters of the alphabet (all of which are terminal symbols).

The curly-bracket and square-bracket-metasymbols allow repetition within the brackets. The bracket-metasymbols indicate repetitions:

{...}	one
{...}*	zero or more
{...}+	one or more
[...]	zero or one.

The enter-here-metasymbol permits repetition where a full cycle is not made. The enter-here-metasymbol causes the initial entry to be just after the enter-here-metasymbol. After initial entry, the expansion continues at the left curly bracket. For example, a procedure call may include an actual parameter list that has one or more parameter expressions, separated by commas.

```
var -> ident ( { , . exp }+)
```

This generates:

```
var -> ident (exp)
var -> ident (exp, exp)
```

etc., but not:

```
var -> ident (, exp)
```

Appendix E Coding Compatible Assembly Language

External procedures

These procedures must be declared with an external procedure statement in the XPL/S main program. They must be coded to be compatible with the calling sequence generated by the compiler. The following examples are given in detail in order to illustrate the interface. However, it is strongly reccommended that in actual practice, the macro library System SYSXPLS be used.

A typical assembly language routine is as follows.

```
            DEF         P
            CSECT       0
B           RES         1
A           RES         1
            CSECT       1
            DATA        B
            DATA        A
P           PSW,15      *0
            LW,1        B
            STW,1       *A
            LI,9        1
            PLW,15      *0
            B           *15
```

The appropriate XPL/S declaration for this routine is:

```
EXTERNAL PROCEDURE P (A,B) BIT(1);
   DECLARE A FIXED REF, B FIXED;
END;
```

It's action is to replace A with the value of B and return the usual representation of true.

Some particular points about the external interface.

(1) Parameters are stored in reverse order above the entry point. In the example, A is the first parameter, B is the second.

(2) Parameters are passed by value unless the formal parameter is declared ref.

(3) Register 15 is used for branch-and-link and register 9 for values returned by functions.

(4) The compiler generates any necessary register save-restore code. No number-to-string conversion is performed.

(5) All parameters are passed as 32-bit quantities.

(6) The comment regarding strings made under external variables should be noted.

(7) Register 0 contains the address of the program's temp stack.

It is set a part of the program initialization.

The access codes on the CSECT directives are specified so as to be compatible with the code generation conventions used by the compiler. That is, the code is given read/execute access only.

A macro that generates the appropriate receiving sequence may be found in System SYSXPLS. Using this macro, the previous example requires only:

```
P       XPROC       A,B
```

External Variables

External procedures may communicate with the XPL/S programs through the use of external variables. These are those for which the storage is allocated within the assembly language procedure and which are declared with the attribute external in the XPL/S program. These are treated exactly as internal variables by the compiler, except that no storage is reserved for them and no initialization is permitted.

Considerable care should be exercised with external character variables with descriptors stored in the assembly procedure. The reason for this is that only those descriptors areas in the active list are examined when it becomes necessary to garbage-collect the free string area. Hence any external string in the free string area may be lost at unpredictable times. However, constant strings are quite safe. The CHAR macro is useful for creating XPL/S compatible constant strings.

A descriptor area may be created within an assembly and added to the list of active descriptor areas so that at garbage collection the strings will survive. The first word of the area should hold the size of the area-1, (the number of descriptors, including nulls).

```
            REF       X:LINK_DESC
AREA        DATA      7
            RES       7
BEGIN       LI,2      AREA
            BAL,15    X:LINK_DESC
```

The code at BEGIN will link AREA so that at garbage collection any strings pointed to by the 7 descriptors will be collected and the descriptors updated properly.

External variables may be placed in blank common. The value in doing this is that it speeds up loading and reduces the space that the absolute program requires on the RAD. An example of this is as follows:

```
      DECLARE (X,Y) FIXED EXTERNAL;

            DEF         X,Y
F4:COM      DSECT
X           RES         1
Y           RES         1
```

Communication into the XPL/S program

Any variable declared global or made global by the DV (def variable) option may be accessed from an assembly language procedure by REFing its name. These are word boundry addresses.

global XPL/S procedures have the same entry protocol as described for external procedures. Hence addresses of global procedure parameters are available to the assembly language procedure. An assembly procedure must protect its registers before calling an XPL/S procedure.

Organization of the load module

The XPL/S compiler follows the standard conventions for manipulation of the GO file. In consequence, other modules may be placed before or after that created for the XPL/S main program. References generated by the compiler will cause the loading of the runtime support module from the library, so care should be taken to avoid the definition of names also defined within it. The main program generated by the compiler defines a transfer address, X:RUN. If this is superseded by another, then it should be used to initiate the main program. In any event, the initialization routine within the runtime support, X:RUN, must be transfered to in order to start the XPL/S code.

Appendix F Storage Organization of XPL/S Programs

Compiler ROM

The relocatable object module (ROM) produced by the XPL/S compiler is divided into four control sections (CSECT). In order of increasing storage address they are: code, data, descriptor and string.

The code CSECT contains the (pure) code generated by the compiler. There are no store instructions directed to this area. This CSECT is given read/execute access.

The data CSECT contains all internal fixed, stack and short bit string variables declared in the program and all the numeric constants whose use requires that a core copy be maintained. The memory cells used by procedures to save the return address are also in this CSECT.

The descriptor CSECT contains the descriptors for all internal character and long bit variables declared in the program and constants used within the program.

The string CSECT contains the text of all character and long bit string constants generated by the program. It contains nothing for those character or long bit variables that are declared without INITIAL. This CSECT is given read-only access.

Runtime support ROM

The runtime support is organized in a similar manner, with the CSECT used for executable code being given read/execute access.

Free string area

All of memory that is available to the XPL/S program but not otherwise allocated is used by the free string area. Its principle function is the storage of strings created in the course of execution of the XPL/S program. It also is used for input-output buffers, if any are requested, and the descriptor sort table used by COMPACTIFY. The free string area is allocated on an increasing address basis until it is full. At this point, the garbage collection routine, COMPACTIFY, is called. (See Appendix G.)

Appendix G Separate Compilations and Overlays

Each XPL/S compilation has one descriptor csect, but all compilations share the same free string area. The first word of each descriptor csect contains the number of descriptors in the csect, including nulls.

At program initialization the address of the descriptor csect for the main compilation, stored at X:MAIN-1, is added to the list of active descriptor areas. When a global procedure is called it passes the address of its compilation's descriptor area to a runtime support procedure that adds it to an ordered list of active descriptor areas, if not there already. If the NL, no link, option is on, the global procedure's receiving sequence is shortened by two words and no descriptor area linking is attempted.

Loading a segment of an overlay will remove from the ordered list all descriptor areas higher in core or overlapping the first word address of the segment. A call to a global procedure in the new segment will add the segment to the active list. There is no roll out of segments, each segment loaded has its compile time data values.

When it is time to compactify the free string area, all areas in the active list are scanned for active strings in the free string area. The concatenate stack used by the runtime support is treated as a descriptor area so that any descriptors in the stack remain valid after the compatification. As strings are moved down in the free string area their descriptors are updated.

As a consequence of the above implementation, segments should be loaded in root to leaf order, a segment may load any descendent, (loading an ancester will remove its own area from the active list) and there is no restriction on the number of XPL/S compilations per segment.